

E-Commerce Backend — Full Plan & System Design Guide

A complete, build-along blueprint for a production-grade e-commerce REST API using the **MERN** stack (Mongo + Express + Node). React frontend is **out of scope** here — this doc is 100% backend.

Use this as a linear guide: each phase produces a runnable slice you can demo with Postman before moving on.

1. Project Overview

What you're building

A multi-role e-commerce backend exposing REST APIs for:

- Browsing products
- Authenticated customer carts & checkout
- Atomic order creation with stock reservation
- Payment flow (mocked via webhook)
- Admin product, inventory, and order management
- Secure file uploads (product images)
- Self-documenting API (Swagger + Postman)

Why this covers everything

Requested Topic	Covered In
ACID & transactions	Section 8 (Checkout)
Stock decrement	Section 7 (Inventory), Section 8
Webhooks mock	Section 9 (Payments)
Idempotency keys	Section 8
bcrypt, JWT, refresh tokens	Section 5
Secure cookies, CORS, Helmet, rate limiter	Section 6
xss / mongo sanitization, CSP	Section 6
Password reset, MFA overview	Section 5
Multer (memory vs disk)	Section 10
Cloudinary/S3 + signed URLs	Section 10
Swagger/OpenAPI + Postman	Section 11
API versioning	Section 3

2. Tech Stack

Layer	Choice	Why
Runtime	Node.js 20+	LTS, native fetch, top-level await
Framework	Express 5	You already know it; middleware ecosystem
Database	MongoDB 7+ (replica set)	Replica set is required for transactions
ODM	Mongoose 8	Schema validation, hooks, sessions
Auth	jsonwebtoken + bcrypt	Access + refresh token pattern
Validation	zod or joi	Pick one; zod gives TS-style inference
File upload	multer	Memory storage → stream to Cloudinary/S3
Media storage	Cloudinary (easier) or AWS S3	Cloudinary for speed of dev
Email	nodemailer + Mailtrap	Password reset, order confirmation
Logging	pino or winston	Structured JSON logs
Docs	swagger-jsdoc + swagger-ui-express	Inline OpenAPI annotations
Testing	jest + supertest + mongodb-memory-server	Integration tests over unit
Security	helmet, cors, express-rate-limit, express-mongo-sanitize, xss-clean, hpp	OWASP basics
Env	dotenv	<code>.env</code> , <code>.env.example</code> committed
Process	pm2 (prod) / nodemon (dev)	

Replica set tip for local dev: use `run-rs` (`npm i -g run-rs`) or `mongodb-memory-server` with `replSet: 'rs0'`. A standalone `mongod` cannot do transactions.

3. System Architecture

Architectural style

Modular monolith with **layered architecture per feature**:

```
routes → middleware → controller → service → model
                                     ↑
                               (business logic lives here)
```

- **Controller:** parse request, call service, format response. No DB calls.
- **Service:** business logic + transactions + cross-model orchestration.
- **Model:** Mongoose schema + instance/static methods only.
- **Middleware:** auth, validation, rate-limiting, error handling.

Folder structure — principles, not a template

Design your own layout. The non-negotiable principles:

- **Feature-first inside `api/v1/`**, not type-first. Each feature folder owns its routes, controller, service, and validator together. (Why? When you change "checkout", you want all the relevant code in one place — not scattered across `controllers/`, `routes/`, `validators/`.)
- **Models in their own folder** (they're shared across features).
- **Cross-cutting middleware** (auth, validation wrapper, error handler, rate limit, idempotency) in `middlewares/`.
- **Config in its own folder** — one file per concern (db, env, clouinary, swagger).
- **Reusable utilities** (ApiError class, asyncHandler, token helpers, logger) in `utils/`.
- **Split `app.js` (builds the Express app) from `server.js` (calls `listen`)** — this split is what makes the app testable with supertest without actually opening a port.
- Mount everything under `/api/v1/` from day one.

Sketch your layout on paper, then justify each folder. If you can't say what goes in it and what doesn't, the folder shouldn't exist yet.

API versioning

- Mount everything under `/api/v1/...`.
- When a breaking change comes, add `/api/v2/...` next to v1 — never modify v1's contract.
- Keep version in the URL (simplest); header-based versioning is also valid but harder to debug in browsers.

Conventions

- Plural resource names: `/products`, `/orders`.
- Use HTTP verbs properly: `GET`, `POST`, `PATCH` (partial), `PUT` (full replace, rare), `DELETE`.
- Standard response envelope:

```
{ "success": true, "data": {...}, "meta": { "page": 1, "total": 42 } }
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "...", "det
```

- Status codes: 200, 201, 204, 400, 401, 403, 404, 409 (conflict — useful for idempotency / stock), 422, 429, 500.

4. Database Design

Entity overview

```
User └─ RefreshToken
      └─ Cart └─ CartItem (embedded)
      └─ Order └─ OrderItem (embedded)
      └─ Payment

Product └─ (inventory fields embedded)
        └─ (image refs from Cloudinary)

IdempotencyKey (standalone, TTL-indexed)
```

What each collection needs to hold

Design these schemas yourself — below are the **fields you'll need and why**, not the syntax.

User — identity + credentials. Think about:

- Email (unique, lowercased — why lowercase? consider Gmail-style dot variations).
- A *hashed* password field that must **not** come back on normal queries (research Mongoose's `select: false`).
- A role for RBAC. Start with `customer` / `admin`; can you justify adding more later?
- Email verification flag (when does it matter? checkout? login?).
- Password reset token + expiry (store the **hash** of the token, not the raw one — why?).
- A `passwordChangedAt` timestamp — useful for invalidating old JWTs.
- Optional: MFA secret (encrypted at rest).

Product — catalogue + inventory.

- SKU and slug (both unique). What's each for?
- Title, description, category — text-searchable.
- Price as **integer minor units** (see Money rule below).
- An `images` array of `{ url, publicId, isPrimary }` — what's `publicId` for in a Cloudinary world?
- `stock` (on hand) vs `reservedStock` (held by in-flight checkouts) — these are different concepts; think about why you need both.
- `isActive` for soft-hiding without deleting.

Cart — one per user. Decide upfront:

- Embedded items vs a separate `CartItem` collection? (Embedded is fine here; *when* would you change your mind?)
- Do you snapshot the price at add-time, or always recompute at read-time? (Trade-off: stale display vs honest pricing.)

Order — immutable record of what happened.

- Human-friendly `orderNumber` separate from `_id`. How will you generate one that's collision-free under concurrency?
- **Snapshot** product fields (`sku`, `title`, `unit price`) into each line item. Why? (Hint: what happens when an admin renames a product 3 months later?)
- A `status` enum that captures the full lifecycle (pending payment → paid → shipped → delivered, plus failure branches).
- Address, totals (subtotal/tax/shipping/total), currency, payment reference, idempotency key, timestamps.

Payment — one per order (in this design).

- Provider name, provider's own payment id, amount, currency, status.
- Keep an audit array of raw webhook payloads received — debugging gold.

RefreshToken — rotation tracking.

- Store the **hash** of the token, never the raw value (treat it like a password).
- A `family` id so reuse of an already-rotated token can revoke the entire chain (this is the theft-detection mechanism — make sure you understand it before implementing).
- TTL index on `expiresAt` so Mongo cleans up expired rows for you.
- Capture `userAgent` + `ip` for a "your active sessions" feature later.

IdempotencyKey — see §8 for the contract this enforces.

- Unique key, requester, endpoint, hash of the request body, the cached response (status + body), TTL of ~24h.

For each schema, before writing it: ask yourself what queries you'll run against it, and let *those* drive your indexes. Don't index speculatively.

Money rule

Store all money as integer minor units (paise/cents). Never `Number` with decimals — floats break accounting. Format only at the edge.

Indexes recap

- Unique: `User.email`, `Product.sku`, `Product.slug`, `Order.orderNumber`, `Cart.user`, `IdempotencyKey.key`.
- Query: `Product.category+isActive`, `Order.user+status`, text on `Product.title`.
- TTL: `RefreshToken.expiresAt`, `IdempotencyKey.createdAt`.

5. Authentication & Authorization

Token strategy: access + refresh

- **Access token:** JWT, HS256 (or RS256 if you want key rotation), `exp = 15m`, payload `{ sub: userId, role }`. Sent via `Authorization: Bearer ...`.
- **Refresh token:** opaque random string (e.g. 64 bytes hex), stored **hashed** in DB, sent only via **httpOnly secure cookie** on `/auth/refresh`. `exp = 7d`. Rotated on every use.

Why both?

- Short-lived access tokens → small blast radius if leaked.
- Refresh token rotation + family tracking → detect token theft (if an already-used refresh token is presented, revoke the whole family).

Endpoints

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh      # reads refresh cookie, rotates
POST /api/v1/auth/logout      # clears cookie, revokes refresh
POST /api/v1/auth/forgot-password # emails reset link
POST /api/v1/auth/reset-password # consumes reset token
POST /api/v1/auth/verify-email
GET  /api/v1/auth/me
POST /api/v1/auth/mfa/setup      # returns QR + secret
POST /api/v1/auth/mfa/verify     # confirms enrolment
```

Password handling

- **bcrypt**, cost factor **12** (bump as hardware improves).
- Hash on the way in; never log the raw password; never `select: true` the hash by default.
- Enforce min 10 chars, reject common passwords (use `zxcvbn` if you want polish).

Password reset flow

1. User POSTs email → generate `crypto.randomBytes(32).toString('hex')`.
2. Store `sha256(token)` + expiry (15 min) on user doc.
3. Email link: `https://app.com/reset?token=<raw>`.
4. On reset, hash incoming token, look up, check expiry, update password, **invalidate all refresh tokens** for that user, set `passwordChangedAt`.

MFA (overview only — implement if time permits)

- TOTP via `otplib` / `speakeasy`. Store `secret` encrypted at rest.
- Login flow becomes 2-step: password OK → `mfa_required` response → user submits 6-digit code → tokens issued.

RBAC

Simple role check middleware:

```
requireRole('admin')           // single role
requireRole(['admin','staff']) // any of
```

For finer control later, evolve to permissions (`product:write` , `order:refund`).

6. Security Hardening

The middleware stack — order matters

You'll register these in `app.js`. The **order** is non-negotiable — figure out for yourself why each one comes where it does (e.g. why must body parsers come *before* sanitizers? why must rate limiters come early?):

- `helmet` — sets a bundle of security headers; know which ones and what each defends against.
- `cors` — whitelist origins from env; understand what `credentials: true` enables and requires.
- `cookieParser` — needed before any route reads cookies.
- Body parsers (`express.json` , `express.urlencoded`) — **always set a limit**. What's a reasonable cap for an API that mostly handles small JSON? (Hint: uploads go through Multer, not these.)
- `express-mongo-sanitize` — what shape of attack does this stop?
- `xss-clean` (or a modern equivalent — check what's maintained) — same question.
- `hpp` — HTTP Parameter Pollution. Look up what this attack actually does before you copy-paste the middleware.
- A global rate limiter.
- **Stricter** per-route limiters on `/auth/login` , `/auth/forgot-password` , `/checkout` .

A useful exercise: deliberately remove one of these mid-development, send the attack it's supposed to block via curl, and confirm the impact.

CORS

- **Whitelist** origins from env. Never `*` when using credentials.

- `credentials: true` is required for cookie auth.

Secure cookie config (refresh token)

When you set the refresh cookie, get every one of these flags right — research what each does and why before turning it on:

- `httpOnly` — what attack class does this block?
- `secure` — should it differ between dev and prod, or be on always?
- `sameSite` — `strict` vs `lax` vs `none`: which one fits a same-origin SPA? A cross-origin one?
- `path` — why scope to `/api/v1/auth` instead of `/`?
- `maxAge` — should it match your refresh-token TTL, or be a bit longer/shorter?

A wrong combination here silently disables your protection — the cookie still works.

CSRF: when do you need it?

- If **all** auth uses `Authorization: Bearer` headers → CSRF is moot (browsers don't auto-send headers cross-origin).
- If you use **cookies** for auth (refresh cookie counts) → protect endpoints that read cookies with one of:
 - `sameSite: 'strict'` cookies (already above) — usually enough.
 - **Double-submit cookie**: server sets a `csrf_token` cookie; client copies it into `X-CSRF-Token` header; server compares.
 - `csrf` is deprecated — use `csrf-csrf` for the modern double-submit pattern.

Demo a CSRF attack in your write-up: a malicious page POSTing a form to `/api/v1/orders` succeeds without protection, fails with `sameSite/strict` + double-submit.

Rate limiting

- Global: 100 req / 15 min / IP.
- `/auth/login`: 5 / 15 min / IP — locks brute force.
- `/auth/forgot-password`: 3 / hour / IP — prevents email spam.
- Per-user limits on `/orders` (e.g. 10 / min) to slow abuse.

Content Security Policy

- Helmet ships a sane default. Override for image domains: `img-src 'self' res.cloudinary.com data: .`
- API-only servers (no HTML) can keep defaults; main value of CSP is for HTML responses.

Other

- Validate **every** input with zod/joi. Reject unknown fields (`strict()` mode).

- Never trust `req.user.role` from the client — only from a verified JWT.
 - Log security events (failed logins, role escalations attempted) at WARN.
 - Rotate JWT secret? Use `kid` header + a key map.
-

7. Core Modules — Detailed

7.1 Users module

- CRUD on own profile (`GET /me` , `PATCH /me`).
- Admin: list users, deactivate, change role.

7.2 Products module

- Public: `GET /products` (paginated, filterable by `category` , `q` , `minPrice` , `maxPrice` ; sortable).
- Public: `GET /products/:slug` .
- Admin: `POST/PATCH/DELETE /admin/products` .
- Pagination: cursor-based for large catalogues, offset for simplicity to start.
- Search: start with Mongo text index; document the upgrade path to Atlas Search / Elastic.

7.3 Cart module

- One active cart per user (upsert on first add).
- `POST /cart/items` — add or increment.
- `PATCH /cart/items/:productId` — set quantity (0 removes).
- `DELETE /cart` — clear.
- **Validate stock at add time** but **don't reserve** here — reservation happens at checkout.

7.4 Inventory

- `stock` = total on hand. `reservedStock` = held by in-flight orders.
 - Available = `stock - reservedStock` . Expose via product API.
 - Admin: `PATCH /admin/products/:id/stock { delta: +50, reason: 'restock' }` — keep an audit log if you want polish.
-

8. Checkout, Transactions & Idempotency (the showcase module)

The problem

Checkout must atomically:

1. Re-validate prices and stock.
2. Decrement stock on N products.
3. Create an Order document.
4. Create a Payment document (status `initiated`).
5. Empty the cart.

If step 3 fails after step 2 succeeds, you've sold inventory you didn't ship → real money lost. **This requires ACID across multiple documents** → Mongo transactions → **replica set required**.

Designing the transactional checkout

Work through this yourself — the implementation is the whole point. The shape of the solution:

1. **Open a Mongoose session** and wrap your writes in a `withTransaction` callback. Every write inside must pass `{ session }` or it won't be part of the transaction (this is the silent bug that catches everyone the first time — write a test that proves your writes are participating).
2. **Decrement stock with a conditional update.** Instead of "read stock → check → write new stock" (which has a race window), use a single atomic `findOneAndUpdate` whose *filter* itself requires `stock >= quantity`. If no document matches, you know stock ran out — return 409. Look up "optimistic concurrency in MongoDB" to understand why this pattern works without explicit locks.
3. **Re-snapshot prices from the DB**, not from the cart. The cart's price is from when the user added the item — it may be stale or even attacker-tampered. Trust the product collection.
4. **Create the order**, then the payment intent (status `initiated`), then link them.
5. **Clear the cart** — last step, so a transaction abort doesn't lose the user's selections.
6. **Always end the session** in a `finally`.

Questions to answer before you start coding:

- How do you generate a unique, human-friendly `orderNumber` *inside* a transaction without a race?
- If the transaction aborts midway, does the stock decrement also roll back? (Test this — don't assume.)
- What status code do you return for "out of stock" — 400, 409, or 422? Justify your pick.

Idempotency keys

Why: network retries, double-clicks, mobile flakiness. Without keys, the same checkout creates duplicate orders + duplicate charges.

The contract you need to enforce (figure out the implementation as middleware):

- Client must send an `Idempotency-Key` header (a UUID it generated) on every `POST /checkout`.
- On the server, the *key alone* isn't enough — you also need to hash the request body and compare. Otherwise a client could reuse a key with a different cart and get the cached

response for the *old* cart.

- Three outcomes when a key arrives:
 1. **Key seen before, same request hash** → replay the stored response verbatim. The client never knows their retry was a retry.
 2. **Key seen before, different request hash** → reject (409 or 422 — pick one and document it). The key has been reused incorrectly.
 3. **Key not seen** → process normally, then *persist the response* (status + body) keyed by the idempotency key.
- TTL the records at ~24h via a Mongo TTL index.

Edge cases worth thinking through:

- What if your handler crashes *between* processing and persisting the response? (Hint: you may want to write the key as "in-flight" first and finalize later.)
- Should GET requests ever need idempotency keys? Why or why not?
- Reuse the same middleware on refunds and any other non-idempotent write — design it generically.

9. Payments (Mock) & Webhooks

Mock provider

Build a fake "provider" endpoint inside your own app under `/mock-provider/charge` that:

- Accepts `{ paymentId, simulate: 'success' | 'failure' }`.
- After 1–3 seconds, POSTs a webhook to your own `/api/v1/payments/webhook`.

Real shape

Your code should behave as if Stripe/Razorpay is calling it:

```
POST /api/v1/payments/webhook
Headers:
  X-Webhook-Signature: <hmac-sha256 of body using shared secret>
Body:
  { "event": "payment.succeeded", "data": { "providerPaymentId": "mock_xyz" } }
```

Webhook handler rules (these are interview gold)

1. **Verify signature first**, then parse — drop unsigned/invalid immediately with 400.
2. **Use the raw body** for signature verification. Mount `express.raw({ type: 'application/json' })` on the webhook route, not the global JSON parser.
3. **Idempotent processing**: store `event.id` and skip if already processed (webhooks retry).

4. **Return 2xx fast.** Do heavy work async (queue / setImmediate) so the provider doesn't time out and re-fire.
5. On `payment.succeeded`: find payment → flip `Payment.status = 'succeeded'`, `Order.status = 'paid'`, clear `reservedStock` (move from reserved to "sold" — i.e. don't touch stock again, but zero out reservation if you tracked it).
6. On `payment.failed`: flip statuses + **restock** — increment `Product.stock` back by the order quantities (in a transaction).

10. File Uploads & Media

Multer: memory vs disk

	Memory storage	Disk storage
Use when	You're streaming straight to S3/Cloudinary	You need OS-level processing (ffmpeg, sharp on huge files)
RAM	Whole file in <code>req.file.buffer</code>	Streamed to tmp file
Risk	OOM on big uploads	Tmp cleanup, disk full

For an e-commerce product image flow, **memory storage** + Cloudinary upload stream is the right default.

Validation (always do this)

When you configure Multer, decide on **every** one of these and justify each number:

- A file size limit (what's reasonable for a product photo?).
- A max file count per request.
- An allowlist of mime types (jpeg, png, webp — anything else?).
- A `fileFilter` that rejects everything not on the allowlist.

Never trust the client-provided `mimetype` alone — it can be spoofed. Research how to sniff a file's actual format from its magic bytes, and add that check before you accept the upload.

Cloudinary flow

1. Admin hits `POST /admin/products/:id/images` (multipart/form-data).
2. Multer parses → `req.files[]` in memory.
3. For each: `cloudinary.uploader.upload_stream({ folder: 'products' }, cb)` piped from `Readable.from(file.buffer)`.
4. Store `{ url, publicId, isPrimary }` on `product.images`.
5. Delete from Cloudinary on product delete (use `publicId`).

Signed URLs (concept to document)

- Public delivery is fine for product photos.
- For private content (invoices, user uploads), generate **time-limited signed URLs** so the asset URL alone isn't enough to access it.
- Cloudinary: `cloudinary.utils.private_download_url(publicId, format, { expires_at: ... })`.
- S3: `getSignedUrl(s3, new GetObjectCommand({...}), { expiresIn: 300 })`.

S3 alternative (if you go that way)

Use `@aws-sdk/client-s3` + `@aws-sdk/s3-request-presigner`. Better for fine-grained access control; more setup (IAM, bucket policy, CORS).

11. API Documentation

Swagger / OpenAPI

- Install `swagger-jsdoc` + `swagger-ui-express` and learn the **JSDoc-style annotation** format — you'll write OpenAPI fragments above each route handler.
- For each endpoint, document: tag (group), summary, path/query/body params with types, all possible response codes, and the response schema.
- Pull shared schemas (User, Product, error envelope) into `components.schemas` once and `$ref` them everywhere — copy-pasting schemas is how docs drift from reality.
- Mount the UI at `/docs`. In production, put it behind basic auth so you're not leaking your API surface to the internet.
- Litmus test: a developer who has never seen your code should be able to hit every endpoint correctly using only `/docs`.

Postman collection

- Maintain a `docs/postman/ecommerce.postman_collection.json`.
 - Use Postman environment variables: `{{baseUrl}}`, `{{accessToken}}`, `{{refreshToken}}`.
 - Use a pre-request script on the collection that auto-refreshes the access token when it expires.
 - Export and commit the collection; export environment **without secrets**.
 - Bonus: generate it from your OpenAPI spec using `openapi-to-postmanv2`.
-

12. Error Handling & Logging

Central error handler

- `ApiError` class with `(status, code, message, details)`.
- `asyncHandler(fn)` wrapper so controllers don't repeat try/catch.
- Final `errorHandler` middleware maps:
 - `ApiError` → its own status + envelope.
 - Mongoose `ValidationError` → 422 with field details.
 - Mongoose `CastError` → 400.
 - Mongo duplicate key (E11000) → 409 with the conflicting field.
 - Anything else → 500, log full stack, return generic message (don't leak internals).

Logging

- `pino` with `pino-http` for request logs.
- Each request gets a `requestId` (uuid) — propagate via `X-Request-Id`.
- Never log: passwords, tokens, full card numbers, full emails in some jurisdictions.
- Log levels: ERROR (5xx + unexpected), WARN (auth failures, 4xx of interest), INFO (startup, important business events), DEBUG (dev only).

13. Testing Strategy

Lean toward **integration tests** that hit real Express routes + real Mongo (memory-server) — they catch the bugs that matter for an API.

Layer	Tool	What to test
Integration	jest + supertest + mongodb-memory-server (replSet)	Full request/response, auth, transactions
Unit	jest	Pure utils (price calculations, token helpers)
Contract	dredd or schemathesis	OpenAPI spec ↔ actual responses

Must-test scenarios:

- Register + login + refresh + logout happy path.
- Login with bad password is rate-limited after N tries.
- Checkout decrements stock atomically (simulate two parallel checkouts for last item — only one wins, other gets 409).
- Idempotency: same key returns the same response, doesn't create duplicate orders.

- Webhook with bad signature is rejected.
 - Product upload rejects oversized / wrong mimetype files.
-

14. Configuration & Secrets

`.env.example` — commit a template, never real values

Maintain a `.env.example` that lists **every** variable your app reads, with placeholder values. Group them by concern:

- Runtime: `NODE_ENV`, `PORT`.
- Database: Mongo connection string (don't forget the `replicaSet` query param for transactions).
- Auth: separate secrets for access and refresh tokens, plus TTLs.
- CORS: comma-separated allowed origins.
- Cookies: domain.
- Cloudinary / S3 credentials.
- SMTP (Mailtrap in dev) for password-reset emails.
- Webhook signing secret.
- Rate-limit window + max.

Two rules to internalise:

- The example file is the contract — every new env var goes in here the same commit it's introduced.
- The real `.env` never goes in git. Make sure `.gitignore` is set up *before* you write any secrets.

Validate env at boot

Don't read `process.env.X` ad-hoc throughout the code. Build a single `config/env.js` module that validates the entire env (use `zod` or `envalid`) and exports a typed config object. The app should **crash loudly at startup** if anything is missing or malformed — never silently fall back to `undefined` and discover the bug in prod.

15. Phased Implementation Plan

Each phase produces a demoable slice. Don't move on until the previous phase's tests pass.

Phase 0 — Skeleton (½ day)

- Init repo, `package.json`, ESLint + Prettier, folder structure, `app.js` / `server.js` split, env validation, `/health` endpoint, central error handler, pino logger.

Phase 1 — Auth core (1–2 days)

- User model + bcrypt.
- Register + login returning access + refresh.
- Refresh rotation + revocation chain.
- `requireAuth` + `requireRole` middleware.
- Postman collection draft.

Phase 2 — Security hardening (½ day)

- Helmet, CORS, rate limit, mongo-sanitize, xss-clean, hpp, body size limits.
- Strict limiter on auth routes.
- Password reset flow (email via Mailtrap).
- Demo a CSRF scenario in your notes; add `sameSite: 'strict'` + (optional) double-submit on cookie routes.

Phase 3 — Products & inventory (1 day)

- Product CRUD (admin) + public listing with filters/pagination/text search.
- Indexes set up.
- Validation with zod.

Phase 4 — Cart (½ day)

- Add/update/remove items.
- Stock check at add time (not yet reservation).

Phase 5 — Checkout, transactions, idempotency (1–2 days, the big one)

- Replica set running locally.
- Transactional checkout service.
- Idempotency middleware + DB model with TTL.
- Concurrency test: two parallel checkouts for the last unit.

Phase 6 — Payments mock + webhooks (1 day)

- Payment model, mock provider route, webhook receiver.
- HMAC signature verification on raw body.
- Idempotent webhook processing.

- Success path → order paid. Failure path → restock in transaction.

Phase 7 — File uploads (½ day)

- Multer memory storage + validation.
- Cloudinary integration on product images endpoint.
- Document signed-URL concept with a working example for one private resource.

Phase 8 — Docs (½ day)

- Swagger annotations across all routes.
- Reusable component schemas.
- `/docs` route (basic-auth in prod).
- Finalize Postman collection + environment.

Phase 9 — Polish (½ day)

- README with architecture diagram + setup steps.
- Curl examples for every endpoint.
- Loom-style screen recording of the demo flow (optional but interview gold).

Estimated total: 7–10 working days at a sustainable pace.

16. Stretch Goals (only after the core is solid)

- MFA (TOTP) end-to-end.
 - Coupons / discount codes (with idempotency on application).
 - Order status webhooks **outbound** to a merchant URL.
 - Background jobs (BullMQ + Redis) for emails and webhook retries.
 - Soft deletes + audit log.
 - Centralized logging to a file with rotation; OpenTelemetry traces.
 - Docker + docker-compose (app + mongo replica set + mailhog).
 - GitHub Actions CI: lint + test on PR.
-

17. Definition of Done (use this as a checklist)

- ☐ All endpoints documented in Swagger and Postman.
- ☐ All `req.body` / `req.query` / `req.params` validated.
- ☐ All write endpoints have appropriate auth + role checks.
- ☐ Checkout is transactional and idempotent — proven by a test.

- ☐ Webhook verifies signature on raw body and is idempotent.
 - ☐ Passwords are bcrypted; refresh tokens are hashed at rest and rotated.
 - ☐ Helmet, CORS whitelist, global + per-route rate limits all active.
 - ☐ All money stored as integer minor units.
 - ☐ `.env.example` committed; secrets never committed.
 - ☐ README documents architecture, setup (including replica set), and demo flow.
 - ☐ Integration tests cover the must-test scenarios in §13.
-

18. Quick Reference: Endpoint Map

Auth

POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
POST /api/v1/auth/verify-email
GET /api/v1/auth/me
POST /api/v1/auth/mfa/setup
POST /api/v1/auth/mfa/verify

Users

GET /api/v1/users/me
PATCH /api/v1/users/me

Products

GET /api/v1/products
GET /api/v1/products/:slug

Cart

GET /api/v1/cart
POST /api/v1/cart/items
PATCH /api/v1/cart/items/:productId
DELETE /api/v1/cart/items/:productId
DELETE /api/v1/cart

Checkout / Orders

POST /api/v1/checkout (Idempotency-Key header required)
GET /api/v1/orders
GET /api/v1/orders/:id

Payments

POST /api/v1/payments/webhook (raw body, signature verified)

Uploads (admin)

POST /api/v1/admin/products/:id/images
DELETE /api/v1/admin/products/:id/images/:publicId

Admin

GET /api/v1/admin/users
PATCH /api/v1/admin/users/:id
POST /api/v1/admin/products
PATCH /api/v1/admin/products/:id
DELETE /api/v1/admin/products/:id
PATCH /api/v1/admin/products/:id/stock
GET /api/v1/admin/orders
PATCH /api/v1/admin/orders/:id/status

System

GET	/health
GET	/docs

(basic-auth in prod)

Build order tip: complete Phases 0–2 end-to-end with full Postman docs *before* touching products. The auth + security foundation is what every other phase depends on, and getting it right early saves rework.